

21. ROS2 URDF Models

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. URDF Overview

The modeling method in ROS is called URDF, which stands for Unified Robot Description Format. It can not only clearly describe the robot's own model but also describe the robot's external environment. URDF model files use XML format.

SolidWorks model to URDF plugin: [sw urdf exporter - ROS Wiki](#)

2. Robot Components

During the process of modeling and describing a robot, we first need to be familiar with the robot's components and parameters. For example, a robot generally consists of four main parts: **hardware structure, drive system, sensor system, and control system**. Some common robots on the market, whether mobile robots or robotic arms, can be decomposed according to these four components.



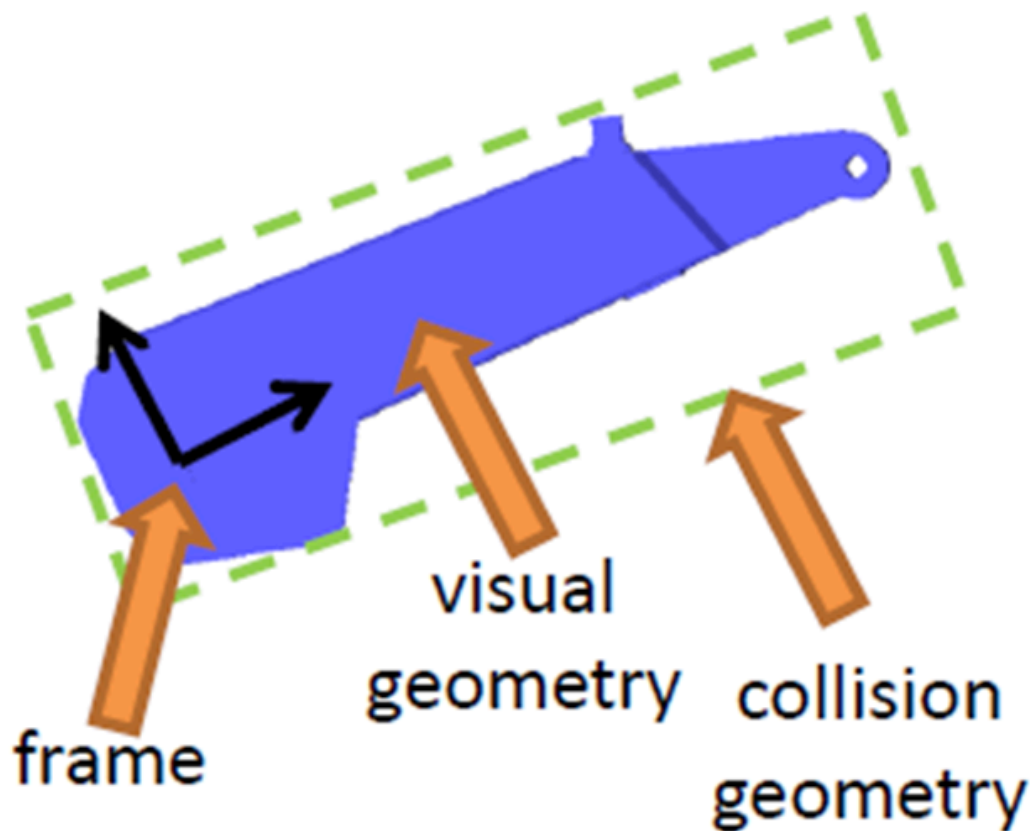
- Hardware structure refers to tangible equipment that can be seen, such as chassis, shell, motors, etc.;
- Drive system refers to devices that enable these equipment to function properly, such as motor drivers, power management systems, etc.;
- Sensor system includes encoders on motors, onboard IMU, installed cameras, LiDAR, etc., which help the robot perceive its own state and external environment;
- Control system is the main carrier during our development process, generally computing platforms like RDK, computers, and the operating systems and application software inside them.

The process of robot modeling is actually similar - through modeling language, describing each part of the robot clearly and then combining them together.

3. URDF Syntax

3.1. Link Description

The `<link>` tag is used to describe the appearance and physical properties of a rigid body part of a robot. Appearance includes size, color, and shape. Physical properties include mass, inertia matrix, collision parameters, etc.



Take this robotic arm link as an example. Its link description is as follows:

```

<link name="link_4">
  <visual>
    <geometry>
      <mesh filename="link_4.stl"/>
    </geometry>
    <origin xyz="0 0 0" rpy="0 0 0" />
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.5" radius="0.1"/>
    </geometry>
    <origin xyz="0 0 -0.05" rpy="0 0 0" />
  </collision>
</link>

```

The name in the link tag represents the link's name. We can customize it, and this name will be used when joints connect links.

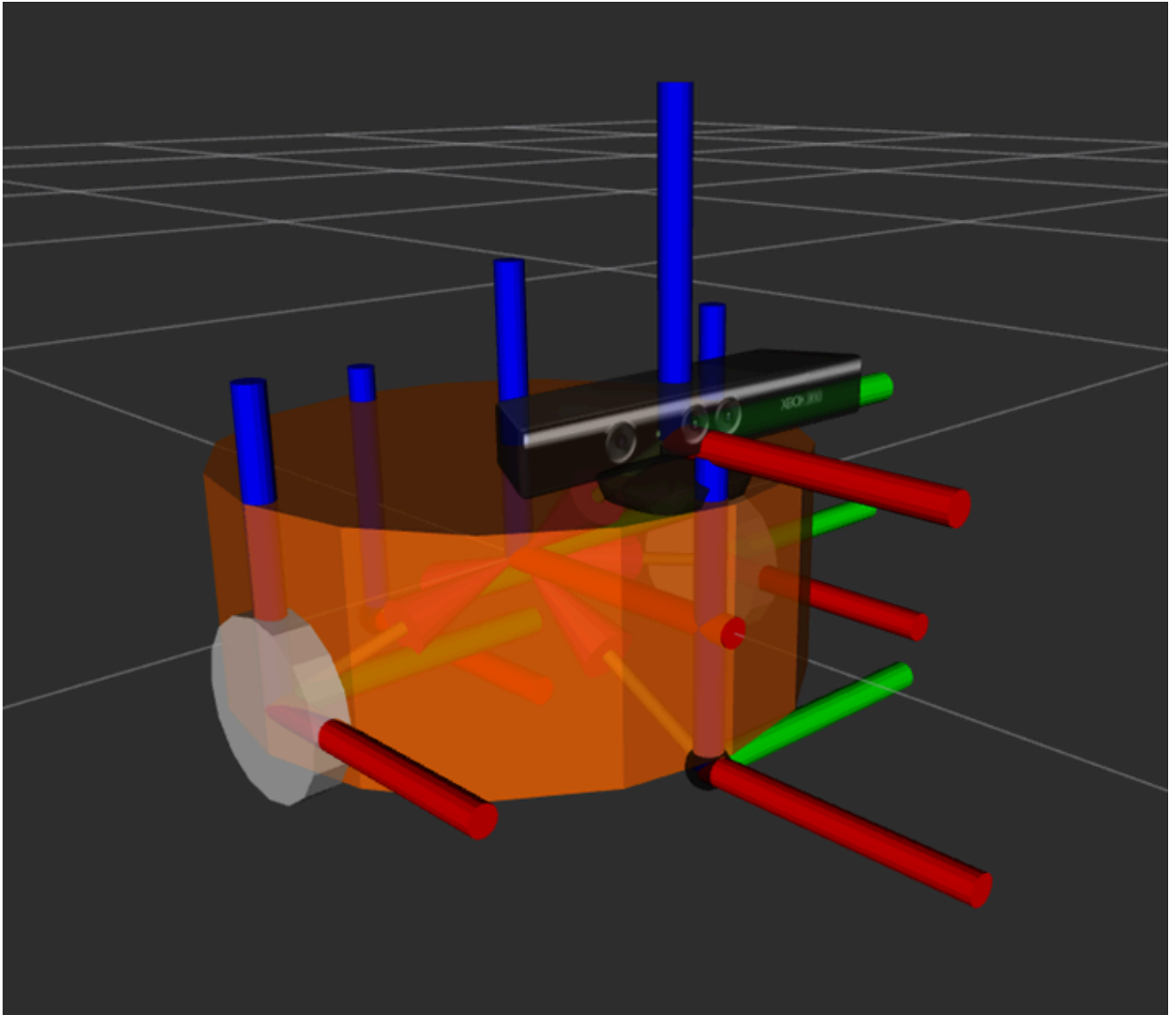
The `<visual>` part inside the link describes the robot's appearance:

- `<geometry>` represents the geometric shape. Inside it, `<mesh>` calls a blue appearance designed in advance in 3D software, which is this stl file. It looks consistent with the real robot.
- `<origin>` represents the offset of the coordinate system relative to the initial position. These are translations in the x, y, z directions and roll, pitch, yaw rotations. If no offset is needed, all are 0.

The second part, `<collision>`, describes collision parameters. The content inside seems similar to `<visual>`, also having `<geometry>` and `<origin>`. They look the same, but the difference is quite significant.

- The `<visual>` part focuses on describing how the robot looks - the visual effect;
- The `<collision>` part describes the robot's state during movement, such as how the robot's contact with the external world is considered a collision.

In this robot model, the blue part is described through `<visual>`. During actual control, this complex appearance requires more computational power for collision detection calculations. To simplify the calculation, we simplified the collision detection model to a green box cylinder, which is the shape described by `<geometry>` inside `<collision>`. The `<origin>` coordinate system offset is similar and can describe the offset of the rigid body's center of mass.



For mobile robots, links can also describe the car's body, wheels, etc.

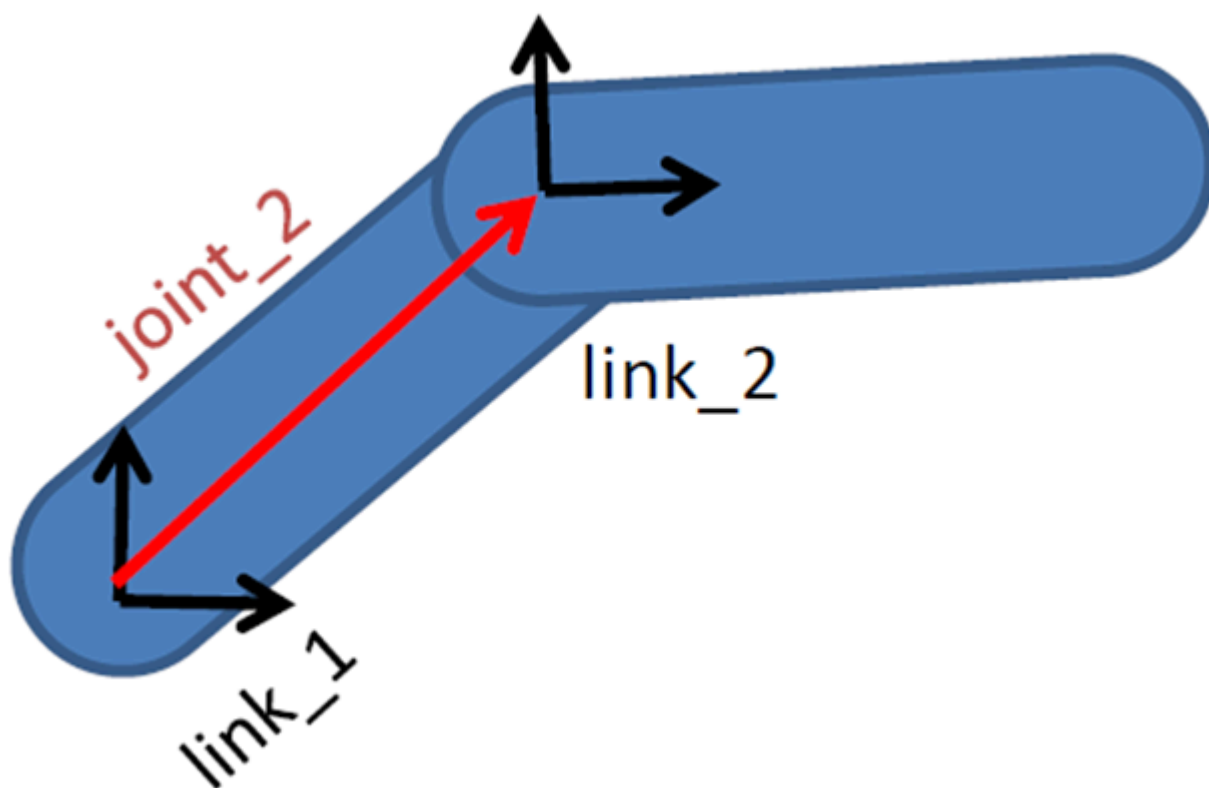
3.2. Joint Description

Rigid bodies in robot models can only produce relative motion after being connected through joints.

There are six types of motion in URDF joints.

关节类型	描述
continuous	旋转关节，可以围绕单轴无限旋转
revolute	旋转关节，类似于continuous，但是有旋转的角度极限
prismatic	滑动关节，沿某一轴线移动关节，带有位置极限
fixed	固定关节，不允许运动的特殊关节
floating	浮动关节，允许进行平移、旋转运动
planar	平面关节，允许在平面正交方向上平移或者旋转

1. continuous: Describes rotational motion that can rotate infinitely around a certain axis. For example, a car's wheels fall into this category.
2. revolute: Also a rotational joint. The difference from continuous type is that it cannot rotate infinitely but has angle limits. For example, the two links of a robotic arm fall into this type of motion.
3. prismatic: Is a sliding joint that can translate along a certain axis and also has position limits. Generally, linear motors use this type of motion.
4. fixed: Is a fixed joint. It's the only type that doesn't allow motion, but it's used quite frequently. For example, the camera link is installed on the robot, and the relative position doesn't change. At this time, the connection method used is Fixed.
5. floating is a floating joint, and the sixth type is planar, which is a planar joint. These two are used relatively less frequently.

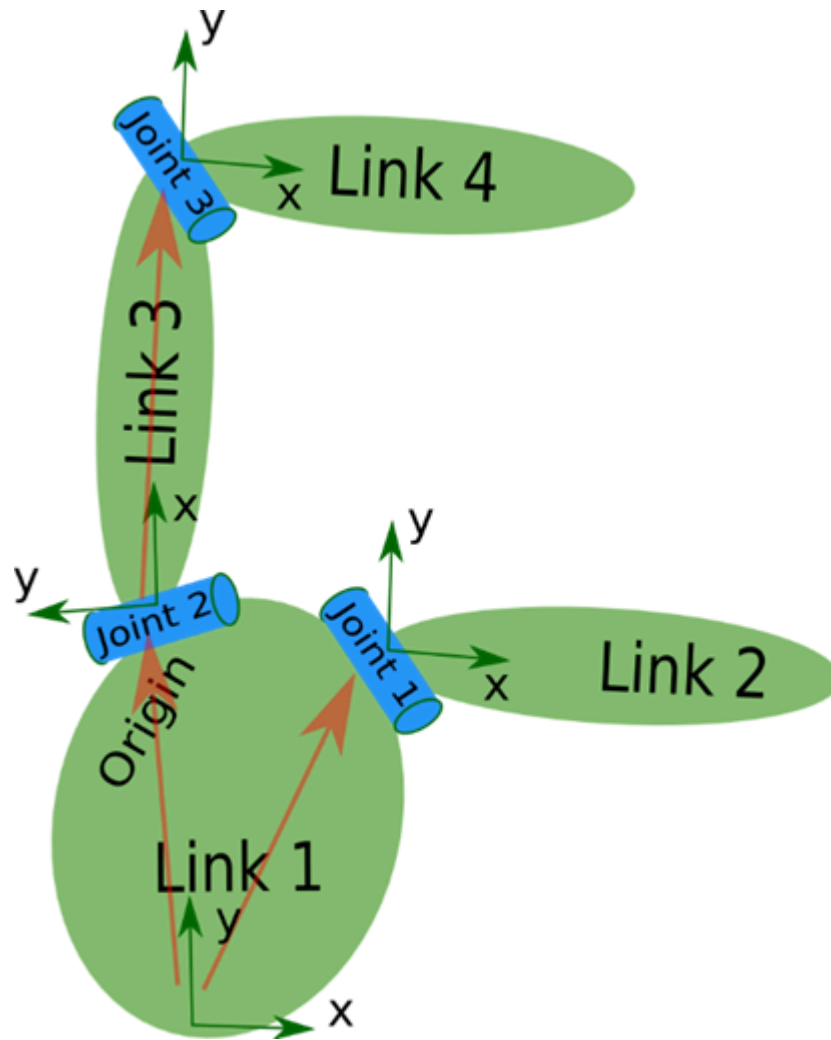


In the URDF model, each link is described with such XML content, such as what the joint is called and what type of motion it is.

```
<joint name="joint_2" type="revolute">
  <parent link="link_1"/>
  <child link="link_2"/>
  <origin xyz="0.2 0.2 0" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-3.14" upper="3.14" velocity="1.0"/>
</joint>
```

- parent tag: Describes the parent link;
- child tag: Describes the child link. The child link will move relative to the parent link;
- origin: Represents the relationship between the two link coordinate systems, which is the red vector in the figure. It can be understood as how these two links should be installed together;
- axis represents the unit vector of the joint's motion axis. For example, if z equals 1, it means this rotational motion is performed around the positive direction of the z-axis;
- limit represents some limitations on the motion, such as minimum position, maximum position, and maximum velocity.

4. Complete Robot Model



Finally, all link and joint tags complete the description and combination of each part of the robot. They are all placed in a robot tag, forming a complete robot model.

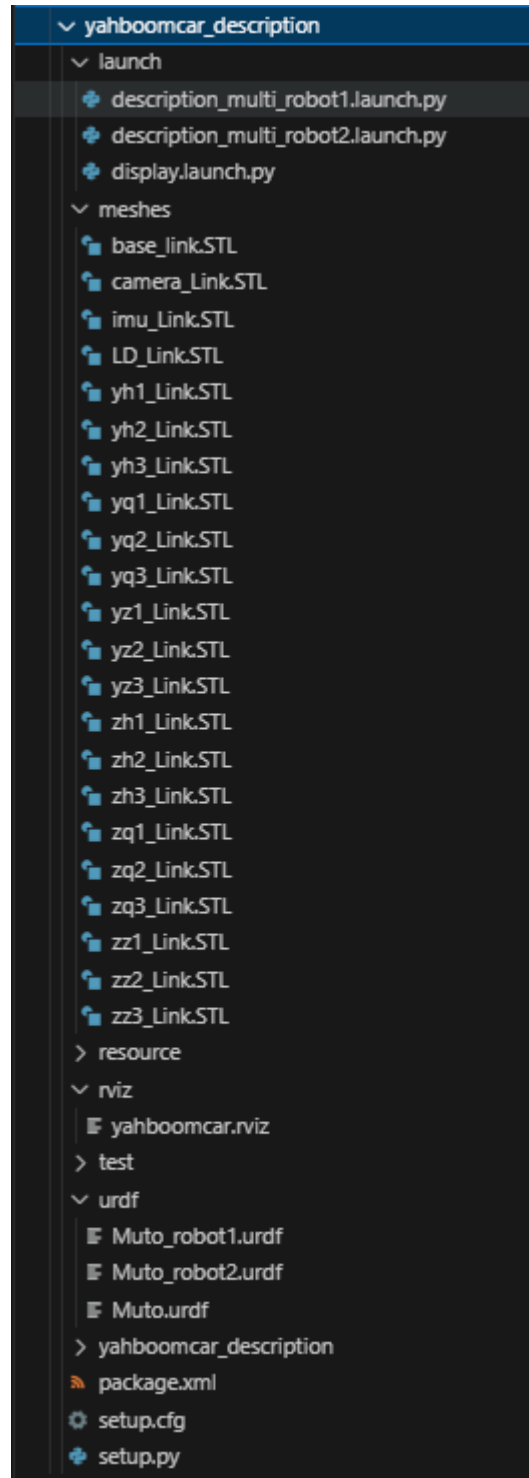
```
<robot name="<name of the robot>">
  <link> ..... </link>
  <link> ..... </link>

  <joint> ..... </joint>
  <joint> ..... </joint>
</robot>
```

So when looking at a URDF model, don't rush to look at the details of each piece of code. First, find links and joints to see what parts this robot consists of. After understanding the overall situation, then look at the details.

4.1. Create Robot Model

Taking muto's model as an example, copy the `yahboomcar_description` package from this section's tutorial folder to the src directory of the workspace:



- `urdf`: Stores URDF or xacro files for robot models
- `meshes`: Stores model rendering files referenced in URDF
- `launch`: Saves related launch files
- `rviz`: Saves rviz configuration files

Then build the package:

```
colcon build --packages-select yahboomcar_description
```

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 110x24
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select yahboomcar_description
Starting >>> yahboomcar_description
/usr/lib/python3/dist-packages/setuptools/dist.py:723: UserWarning: Usage of dash-separated 'script-dir' will
not be supported in future versions. Please use the underscore name 'script_dir' instead
  warnings.warn(
/usr/lib/python3/dist-packages/setuptools/dist.py:723: UserWarning: Usage of dash-separated 'install-scripts'
will not be supported in future versions. Please use the underscore name 'install_scripts' instead
  warnings.warn(
--- stderr: yahboomcar_description
/usr/lib/python3/dist-packages/setuptools/dist.py:723: UserWarning: Usage of dash-separated 'script-dir' will
not be supported in future versions. Please use the underscore name 'script_dir' instead
  warnings.warn(
---
Finished <<< yahboomcar_description [0.72s]

Summary: 1 package finished [0.92s]
1 package had stderr output: yahboomcar_description
yahboom@yahboom-virtual-machine:~/workspace$
```

4.2. Model Visualization Effect

- Refresh environment variables, then run the launch command:

```
ros2 launch yahboomcar_description display.launch.py
```

This will automatically start rviz and display the robot model:

